

Local-to-Global Software Architecture

A SessionStream-Centered Path from Universal Properties to Presheaves, Sheaves, Topoi, and Cohomology

Custom study text

2026-08-15

Table of Contents

How to use this book

This is a custom mathematics text for a software engineer studying Robert Goldblatt's *Topoi: The Categorical Analysis of Logic* and simultaneously developing sessionstream. It is not a replacement for Goldblatt and it does not attempt to reproduce his book. It borrows a pedagogical attitude from it: introduce a categorical construction through a concrete universal problem, state the arrow-theoretic definition, work examples, then make the reader prove small facts before moving to more abstract structure.

The assumed starting point is roughly Goldblatt Chapter 3, around limits and colimits. You should already be willing to read a diagram, compose arrows, and accept that a universal property can characterize an object without talking about its internal set-theoretic representation. Everything else needed here is developed as it becomes useful.

The recurring case study is sessionstream, understood from the Architecture Garden study at commit fb6b70d62915874e3d3cb9c0b1557814e638ac68. The system has typed commands, a session-scoped hub, canonical backend events, an append-only event path, UI and timeline projections, durable materialized entities, snapshots at an ordinal cut, and snapshot-before-live WebSocket hydration. Its open correctness obligations include per-session serial application, stable redelivery identity, atomic projection progress, deterministic replay, and consistent SQLite snapshot cuts.

The central mathematical question of this text is:

When many components each hold a correct *partial* account of a system, what does it mean for those accounts to fit together into one coherent global account, and what can we learn when they do not?

That question will lead us from limits to presheaves, sheaves, topoi, and finally elementary cohomological diagnostics.

A study rhythm

For each chapter:

1. Read the motivating SessionStream problem before the formal definition.
2. Translate every new mathematical object into a software object yourself before reading the supplied translation.
3. Do at least the first three exercises without looking at the hints.
4. Keep a notebook with two columns: **mathematical structure** and **software witness**.
5. When an exercise asks for a counterexample trace, write an actual event/cursor/snapshot trace, not merely prose.
6. Revisit the same invariant after later chapters. A good invariant often admits several mathematical descriptions.

You will see a repeated ladder:

data → arrows → diagrams → contexts → local sections → gluing → obstructions.

The goal is not to make production code use category-theory vocabulary. The goal is to improve the models in your head so that API boundaries, transactional cuts, replay semantics, and distributed consistency become easier to state precisely.

Part I - Universal properties as architecture

1. Stop asking what an object is made of

A programmer naturally defines things by representation. A Snapshot is a struct. A timeline entity is a protobuf message. A store is a Go interface backed by SQLite. Category theory trains a different reflex: characterize an object by the network of arrows into and out of it.

This is Goldblatt's central move in the chapters leading to limits. Instead of asking whether an object *is literally* some particular set, ask what morphisms it supports and what universal problem it solves.

For software architecture, this is useful because representation changes while contracts survive.

Suppose two implementations both claim to provide a snapshot at ordinal n . One stores materialized rows in SQLite. Another replays the event log on demand. If clients cannot distinguish them through the promised operations, their internal representations are secondary. The more stable specification is behavioral.

1.1 Objects and arrows in a SessionStream category

We can invent many categories from the same software. There is no single metaphysically correct "category of SessionStream." A category is a modeling choice.

For example, define a category C_{view} :

- objects are information-bearing views of one session;
- an arrow $A \rightarrow B$ means “there is a lawful way to derive/forget information from view A to obtain view B ”;
- identities do nothing;
- composition means perform derivations in sequence.

Candidate objects include:

- complete canonical event history up to n ;
- timeline materialization up to n ;
- snapshot at cut n ;
- UI event stream up to n ;
- a client’s reconstructed state.

An arrow need not correspond to a single Go function. It can describe a mathematical contract that several functions jointly implement.

1.2 Isomorphism is “same information up to reversible translation”

Two objects A and B are isomorphic when there are arrows

$$f : A \rightarrow B, g : B \rightarrow A$$

with

$$g \circ f = 1_A, f \circ g = 1_B.$$

For software, this is stronger than “these JSON shapes are similar.” It says neither representation loses information relative to the other.

A protobuf message and its lossless binary serialization may be modeled as isomorphic at an appropriate abstraction level. A full canonical event and a UI projection generally are not: projection intentionally forgets information.

This distinction will matter constantly. Sheaf restriction maps usually *forget* information, so they are almost never isomorphisms.

Exercises

1. Define a category whose objects are event prefixes $H_0, H_1, \dots$ of one fixed session and where there is one arrow $H_n \rightarrow H_m$ exactly when $m \leq n$. Explain why composition exists automatically.
2. Give one plausible SessionStream pair that should be modeled as isomorphic and one pair that definitely should not be.
3. Why is “event ordinal” not generally isomorphic to “logical event identity”? Construct a retry trace showing the information lost if you identify them.
4. Advanced: define a category of schemas where morphisms are backward-compatible decoders. What would an isomorphism mean there?

Hints

For Exercise 1, arrows point from more information to less information. This direction will later make presheaves feel natural.

2. Products, equalizers, pullbacks: joining evidence without lying

Goldblatt develops products, equalizers, and pullbacks before abstract limits because they are reusable universal patterns. For software engineering, these are not ornamental examples. They describe common ways of combining information.

2.1 Product: retain two independent views

A product $A \times B$ comes with projections

$$\pi_A: A \times B \rightarrow A, \pi_B: A \times B \rightarrow B$$

such that any object X equipped with arrows $f: X \rightarrow A$ and $g: X \rightarrow B$ has a unique mediating arrow

$$\langle f, g \rangle: X \rightarrow A \times B.$$

The product is therefore the universal way to carry both pieces of information.

SessionStream already suggests product structure when one canonical event is interpreted by independent projectors. If P_T derives timeline output and P_U derives UI output, their product interpretation records both:

$$(P_T \otimes P_U)(s, e) = (P_T(s, e), P_U(s, e)).$$

The crucial word is *independent*. If the UI projector secretly mutates state consumed by the timeline projector, the clean product model is false.

2.2 Equalizer: select inputs on which two computations agree

Given parallel arrows

$$f, g: A \rightrightarrows B,$$

an equalizer is an arrow $e: E \rightarrow A$ selecting precisely the part of A on which f and g agree, universally among such selections.

Software translation:

An equalizer is a principled “restrict to states satisfying this equality” construction.

Suppose you can compute a timeline digest from two sources:

$$\begin{aligned} f &= \text{digest}(\text{rebuildFromEvents}), \\ g &= \text{digest}(\text{storedMaterialization}). \end{aligned}$$

The equalizer identifies histories/store states for which replay agrees with durable materialization.

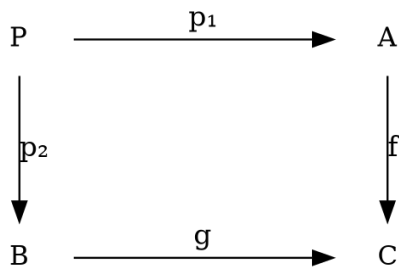
That is close to how executable refinement checks are often organized: build two interpretations and restrict attention to traces where their observations agree.

2.3 Pullback: join two views over a shared coordinate

A pullback of

$$A \xrightarrow{f} C \xleftarrow{g} B$$

is an object P with arrows to A and B whose images in C agree, universal among all such compatible pairs.



A pullback square

In Set, the pullback is concretely

$$A \times_C B = \{(a, b) \in A \times B \mid f(a) = g(b)\}.$$

This is the mathematical version of a typed join on a common key.

SessionStream example: joining snapshot data and cursor evidence

Let

- A be possible snapshot entity sets;
- B be possible snapshot cursor records;
- C be event-prefix coordinates;
- $f(a)$ be the greatest event ordinal represented by entity set a ;
- $g(b)$ be the cursor claimed by b .

A naive equality pullback would demand exact equality. A more faithful SessionStream condition is an inequality:

$$\text{lastEventOrdinal}(x) \leq \text{SnapshotOrdinal}$$

for every returned entity x . We can encode that by changing C from a set of numbers to a relation/poset context, or by defining a predicate object of admissible pairs. The lesson is that

the *shape* of a pullback is often right even when the compatibility relation is richer than equality.

2.4 Why database transactions feel like higher-order witnesses

If you read a cursor and entity rows in separate SQLite operations, you may obtain two locally valid values that were never simultaneously true. A read transaction gives a common witness: both observations came from one database snapshot.

Categorically, transactions often create a stronger object from which several projections originate:

$$\text{transaction snapshot} \rightarrow \{ \text{cursor}, \text{entity set} \}$$

This does not automatically make every transaction a categorical product or pullback. But the universal-construction habit is useful: ask whether several outputs are supposed to be projections of one common witness, and whether your implementation actually constructs that witness.

Exercises

1. Model (UIProjection, TimelineProjection) as a product-like construction. State exactly which purity/noninterference assumptions are required.
2. Define two computations whose equalizer expresses deterministic replay for SessionStream.
3. Write a concrete race where SnapshotOrdinal = 42 but a returned entity has LastEventOrdinal = 43. Which observations are locally valid? Why is the pair globally invalid?
4. REST exercise: an endpoint accepts (orderId, amount) while the invariant depends on (price, quantity, currency). Define a pullback-style compatibility relation between request data and authoritative order state.
5. Prove in Set that the pullback set above has the required universal property.

3. Limits: one coherent answer to a whole diagram

A diagram is a functor from an indexing category J into a category C . Informally, it is a shaped collection of objects and arrows.

A cone from N to a diagram D provides arrows from N to every object of the diagram, compatible with every arrow already in the diagram. A **limit** is a universal cone: every other compatible cone factors uniquely through it.

This is the point in Goldblatt Chapter 3 where products, equalizers, pullbacks, and terminal objects become instances of one idea.

For software, read a limit as:

the most general object that simultaneously provides all the requested views while satisfying all compatibility equations in the diagram.

3.1 A SessionStream limit diagram

Imagine the following requested observations for a session:

- event prefix E_n ;
- timeline state T_n ;
- snapshot S_n ;
- client reconstruction C_n .

There are intended arrows

$$E_n \rightarrow T_n, T_n \rightarrow S_n, S_n \rightarrow C_n,$$

and perhaps another path

$$E_n \rightarrow U_n \rightarrow C_n$$

through UI observations.

A globally coherent execution witness is something that maps into all these views and makes the diagram commute.

If two paths to ClientState disagree, no cone can witness the claimed architecture without weakening some arrow or changing the data.

This “whole diagram at once” perspective is the conceptual predecessor of sheaf gluing.

3.2 Limits are not merely collections

A struct containing five fields is not automatically the limit of five views. A limit includes compatibility.

This is why the distinction between product and pullback matters:

$$A \times B$$

stores arbitrary pairs, whereas

$$A \times_C B$$

stores only compatible pairs over C .

In distributed architecture, a large amount of correctness lives in that subscript $\square_C$.

Exercises

1. Identify terminal-object, product, equalizer, and pullback diagrams as special cases of a limit.

2. Draw a diagram containing event log, projection cursor, materialized state, and snapshot cut. Annotate every arrow with the contract it means.
3. State a candidate limit object for that diagram. Does the current SessionStream implementation actually materialize that object atomically? Explain.
4. Suppose you add an audit projection. Under what conditions can it be added as another leg of an existing limit without changing the previous components?

4. Functors and naturality arrive early

Goldblatt formally develops functors much later than limits. For this custom route we pull them forward because presheaves *are* functors.

A functor $F : C \rightarrow D$ maps objects to objects and arrows to arrows, preserving identities and composition.

For software engineers:

A functor is a translation that respects the structure you decided matters.

A serializer that maps composition of migrations to composition of encoded migrations may be functorial. A projector mapping event histories to view transitions may be functorial under suitable definitions. A logging adapter that reorders operations probably is not functorial with respect to a sequencing category.

4.1 The event-prefix functor

Fix a session s . Let P_s be its prefix poset:

$$0 \leq 1 \leq 2 \leq \dots$$

regarded as a category with one arrow $m \rightarrow n$ whenever $m \leq n$.

A timeline fold can be viewed as assigning a state T_n to every prefix and an evolution map for every prefix extension. Depending on arrow orientation, this is naturally covariant as evolution or contravariant as observation/restriction.

That choice of orientation is not clerical. It expresses whether arrows mean “advance computation” or “forget down to a smaller context.”

Sheaf theory overwhelmingly uses the second orientation.

4.2 Natural transformations are implementation-independent adapters

Given functors $F, G : C \rightarrow D$, a natural transformation $\eta : F \Rightarrow G$ provides an arrow

$$\eta_X : F(X) \rightarrow G(X)$$

for every object X , satisfying

$$G(f) \circ \eta_X = \eta_Y \circ F(f)$$

for every $f : X \rightarrow Y$.

The commuting square says it should not matter whether you translate first and then follow the structure, or follow the structure first and then translate.

Software interpretation

Suppose $F(n)$ is a protobuf timeline representation at prefix n , while $G(n)$ is a JavaScript domain representation. A family of decoders η_n is natural if decoding commutes with restriction to earlier prefixes.

If it does not, your decoder is not merely a format conversion; it is introducing time-dependent semantics.

This makes naturality a useful test for “adapter that should not change meaning.”

Exercises

1. Define a category of prefix cuts and two functors representing backend and browser views. State what a natural transformation between them would mean.
2. Give a non-natural adapter example involving `time.Now()`.
3. Why is a schema migration expected to be natural with respect to some operations but not necessarily all operations? Name the structure you would choose to preserve.
4. Prove that the identity natural transformation is natural.

Part II - Presheaves: software as overlapping contexts

5. Contexts, restrictions, and the presheaf move

We now make the decisive move.

Let C be a category of **contexts**. An arrow $V \rightarrow U$ means that V is a smaller/less informative context contained in or accessible from U .

A presheaf of sets is a functor

$$F : C^{op} \rightarrow Set.$$

Equivalently, for every context U we get a set $F(U)$ of possible observations or states over U , and every inclusion $V \rightarrow U$ induces a restriction

$$\rho_V^U : F(U) \rightarrow F(V).$$

Restrictions obey

$$\rho_U^U = 1_{F(U)}$$

and

$$\rho_w^V \circ \rho_V^U = \rho_w^U.$$

Software translation:

If you forget information in one step or several steps, you must get the same result.

This deceptively simple requirement is the algebra of coherent forgetting.

5.1 Why contravariant?

If $V \subseteq U$, then U is the larger region. But data moves from larger to smaller:

$$F(U) \rightarrow F(V).$$

The inclusion arrow goes one direction; restriction goes the other. Hence $o p$.

For a programmer, op often means “queries travel backward along containment.”

5.2 First SessionStream presheaf: prefix histories

Take contexts to be event-prefix cuts $0, 1, \dots, n$, ordered by inclusion of history.

Let

$$F(n) = \{ \text{valid event histories through cut } n \}.$$

For $m \leq n$, define

$$\rho_m^n(H_n) = H_m$$

by truncation.

Restriction composition is ordinary truncation:

$$\text{truncate}_k(\text{truncate}_m(H_n)) = \text{truncate}_k(H_n).$$

This is a presheaf, although a boring one. Time alone is mostly one-dimensional and totally ordered. The topology becomes more interesting when contexts include *which observer or subsystem* sees the state.

5.3 Observation contexts

Define a context as

$$U = (s, n, K)$$

where

- s is a SessionId,
- n is a cut,
- K is a set of observable dimensions such as event log, timeline, snapshot, UI stream, or client state.

A smaller context can mean either:

- earlier cut $m \leq n$, or
- fewer observable dimensions $K' \subseteq K$, or both.

Now we obtain a genuinely multidimensional poset:

$$\text{Session} \times \text{Time} \times \text{ObserverMask}.$$

A section $x \in F(U)$ is a locally valid assignment to everything visible in U .

Restriction can:

- drop projection fields;
- truncate later events;
- forget payload details;
- retain only a cursor;
- retain only semantic identity;
- map a backend entity to a browser DTO.

The requirement that restrictions compose says all these forgetting paths must agree.

5.4 Hidden coordinates as presheaf bugs

Suppose a projection is claimed to be a function of

$$(event, priorView)$$

but it also reads the wall clock. Two runs with the same declared context may produce different values.

The mathematical diagnosis is not yet “cohomology.” Your proposed presheaf has the wrong base/context or wrong fiber. The clock observation is an unmodeled coordinate.

You have two choices:

1. add the clock observation to the context; or
2. redesign the projector so that the relevant time enters as canonical event data.

This is a powerful design question:

What coordinates must be present so that a local state really is determined by its declared context?

Exercises

1. Define three restriction maps from a rich SessionObservation to smaller views. Verify the identity and composition laws.
2. Construct a restriction diamond with two different paths that should yield the same SnapshotOrdinal. What implementation bug would make the diamond fail?
3. Extend the context tuple (s, n, K) with a schema version. Explain when this is necessary.

4. Add logical EventId as a coordinate. Show how it changes the modeling of retries.
5. Design a presheaf for REST requests where contexts are subsets of parameters and $F(U)$ is the set of locally valid assignments to those parameters.

6. Fibers, sections, stalk-like thinking, and API sufficiency

For a region/context U , $F(U)$ is often called the set of **sections over** U . A section is simply one possible coherent assignment on that context.

This terminology becomes useful because it separates:

- the context itself;
- all possible data over the context;
- one actual assignment.

In software terms:

- context = fields/resources/time window you can observe;
- $F(U)$ = all assignments satisfying local validation;
- section = one request/snapshot/trace instance.

6.1 Parameter sufficiency as fibers of restriction

Suppose X is the full information required to characterize a transaction, while $P \subseteq X$ is the API parameter context. Restriction gives

$$r_p: F(X) \rightarrow F(P).$$

For a concrete request $p \in F(P)$, its fiber is

$$r_p^{-1}(p) = \{x \in F(X) : r_p(x) = p\}.$$

Three cases matter:

1. **empty fiber**: the supplied parameters cannot extend to any globally valid transaction;
2. **singleton fiber**: the parameters determine one complete transaction state;
3. **many-element fiber**: the parameters leave global state underdetermined.

But most APIs do not need to determine the whole world. They need to determine an invariant I . Then parameter sufficiency means:

$$I(x) = I(y) \text{ for all } x, y \in r_p^{-1}(p).$$

So the invariant must be constant on the fiber.

This is a more precise version of “do these parameters contain enough information?”

6.2 SessionStream example: stable retry identity

If the API admits an event based only on

$$(SessionId, Ordinal, Payload),$$

then two bus deliveries of the same logical event that receive different ordinals occupy different points in the declared context. If the invariant is “one logical event is accepted once,” the available coordinates may be insufficient.

Adding stable EventId refines the context so that retries can lie in the same logical fiber.

This is an instance of a general software rule:

An invariant that distinguishes cases your parameters cannot distinguish cannot be enforced solely from those parameters.

Exercises

1. For a payment request with orderId, amount, and optional orderVersion, construct fibers under two assumptions: immutable orders versus versioned orders.
2. State the SessionStream duplicate-delivery invariant as a function I . Explain why (SessionId, Ordinal) is not obviously sufficient to make I constant on retry fibers.
3. Give an example where a parameter set does not determine global state but does determine the invariant you care about.
4. Write a property-based test strategy for checking empirical fiber constancy over generated completions.

7. Covers and nerves: where the multidimensional shape appears

A sheaf needs more than contexts and restrictions. We need a notion of **cover**: a family of local contexts whose union is intended to account for a larger context.

In ordinary topology, open sets U_i cover U when

$$U = \bigcup_i U_i.$$

In software, a cover can mean:

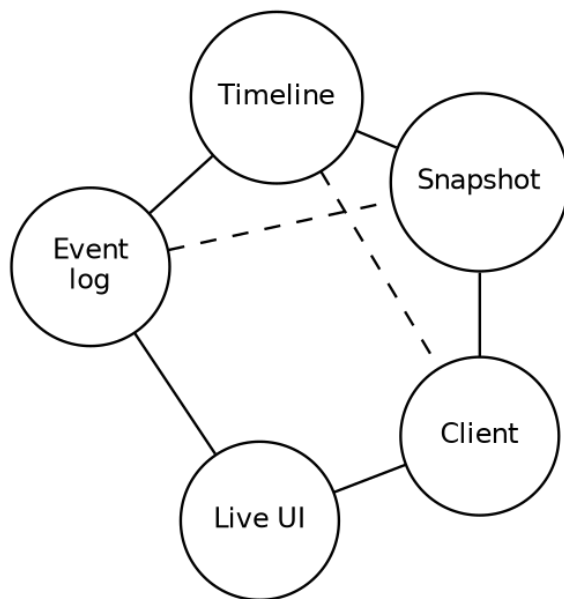
- a set of services whose combined observations account for a transaction;
- several projections that collectively expose a semantic state;
- snapshot plus live suffix covering a client-visible history;
- replicas covering a dataset;
- API parameters and authoritative lookups covering the information needed for an invariant.

7.1 The nerve of a cover

Given a cover $\{U_i\}$, form a combinatorial shape:

- one vertex for each U_i ;
- an edge when $U_i \cap U_j$ is meaningful/nonempty;
- a filled triangle when $U_i \cap U_j \cap U_k$ is meaningful;
- a tetrahedron when four overlap jointly;
- and so on.

This is the **nerve**.



A possible SessionStream observation complex

This is one source of the “multidimensional topological-esque shape” intuition.

A dimension counts how many contexts can participate in one joint overlap:

- 0-simplex: one local observer;
- 1-simplex: pairwise overlap;
- 2-simplex: three-way overlap/witness;
- 3-simplex: four-way overlap/witness.

The geometry is not physical. It is the geometry of **joint knowability**.

7.2 Missing faces matter

Suppose four components form a cycle of pairwise overlaps:

A ---- B
 | |
 | |
 D ---- C

The boundary is present, but no context sees enough to fill the interior with a stronger joint witness.

This is the first place where “hole” becomes an architectural metaphor with mathematical teeth. Pairwise checks can all exist while no higher-order context certifies the whole cycle.

A transaction, coordinator, common log record, or proof object can sometimes add such a witness. Again, do not literalize this too much: the right simplicial model depends on which overlaps genuinely exist.

Exercises

1. Build a nerve for EventStore, TimelineProjection, SnapshotStore, WebSocket hydration, and Client reconstruction. Justify each edge.
2. Which triples have a genuine common witness in the current architecture? Which triangles would you refuse to fill?
3. Add a single database transaction joining projection apply and cursor advancement. Explain how the nerve/context structure changes.
4. Draw the nerve of three REST endpoints that share only an orderId. What semantic information actually lives on the pairwise overlaps?

Part III - Sheaves: local data that really glues

8. The sheaf condition

A presheaf becomes a sheaf when compatible local sections glue uniquely.

Suppose $\{U_i\}$ covers U , and we have sections

$$s_i \in F(U_i).$$

They are **compatible** when every pair agrees after restriction to its overlap:

$$s_i|_{U_i \cap U_j} = s_j|_{U_i \cap U_j}.$$

The sheaf condition says there is a unique global section

$$s \in F(U)$$

whose restriction to each U_i is s_i .

Existence means local consistency is enough to assemble something global.

Uniqueness means the local pieces determine the global result without hidden ambiguity.

Goldblatt's treatment of sheaves emphasizes exactly this compatible-family condition and observes that the global section object can be characterized as a limit of the local-section/overlap diagram. That connection is why studying limits before sheaves is not accidental.

8.1 A sheaf is not “a distributed hashmap”

The sheaf axiom is a semantic statement. It does not say values are physically copied. It says the information model supports lawful local-to-global reconstruction.

You can have a centralized database and a bad presheaf if different query contexts do not compose coherently.

You can have a distributed system with a useful sheaf if all local observations carry enough compatibility information to glue.

8.2 Failure modes

For a compatible family, three situations are possible:

1. **no global extension** - the local pieces cannot all be true of one global state;
2. **multiple extensions** - the pieces underdetermine the global state;
3. **unique extension** - the sheaf property holds for this family.

These correspond directly to software failure modes:

- contradiction;
- ambiguity;
- coherent reconstruction.

Exercises

1. Restate the sheaf condition without using the words “open set,” “topology,” or “continuous.”
2. Give a software example of compatible pairwise projections with two possible global completions.
3. Give an example with no global completion.
4. Explain why uniqueness matters for replay and not merely existence.
5. Show that the sheaf condition is a limit statement over the diagram of cover pieces and overlaps.

9. Snapshot-before-live as a gluing protocol

SessionStream's clearest sheaf-shaped protocol is snapshot-before-live.

The intended history is

$$H_m = e_1 e_2 \cdots e_n e_{n+1} \cdots e_m.$$

A snapshot at cut n represents the prefix

$$H_n = e_1 \cdots e_n,$$

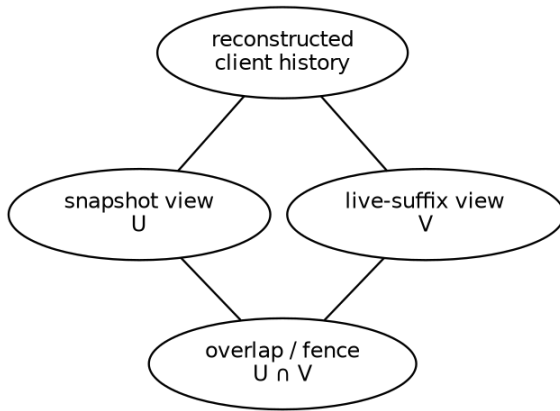
while live delivery provides a suffix strictly after the cut:

$$L_{n,m} = e_{n+1} \cdots e_m.$$

Reconstruction requires

$$\text{fold}(S_n, L_{n,m}) = S_m.$$

The Architecture Garden notes that the WebSocket adapter registers a hydrating subscription, buffers concurrent UI batches while the snapshot loads, sends the snapshot, filters buffered batches at or before SnapshotOrdinal, flushes the later ones in order, and only then marks the subscription live.



Snapshot and live suffix as local pieces glued at a fence

9.1 Choosing the cover

Let U denote the whole client-visible interval from session start through current live state.

Choose two cover pieces:

- U_p : information represented by the snapshot/prefix;
- U_L : information represented by registration-time and future live delivery.

Their overlap is not necessarily an event duplicated on both sides. The overlap can instead be represented by **boundary data**: session identity, snapshot ordinal, registration fence, schema version, and rules defining which side owns an event at the boundary.

This is important. In software sheaf models, overlaps are often semantic interfaces rather than literal duplicated rows.

9.2 Compatibility equations

A candidate compatible family should satisfy at least:

$$\begin{aligned} SessionId_p &= SessionId_L, \\ minOrdinal(L) &> SnapshotOrdinal(P), \end{aligned}$$

and suffix batches must be strictly ordered according to the stream contract.

For accepted live batches after subscription registration, each must be one of:

1. represented in the loaded snapshot;
2. delivered exactly once after the cut;
3. converted into an explicit overflow/reconnect outcome.

This is the Garden's snapshot-plus-suffix completeness law expressed as a gluing condition.

9.3 The hydration buffer is constructive gluing

Mathematics often says “there exists a unique glued section.” Software must construct it under concurrency.

The hydration buffer is operational evidence for the existence part:

- it keeps events that arrive while one local piece is being obtained;
- the ordinal fence decides their restriction/ownership at the overlap;
- ordered flushing creates the combined history.

An overflow is not merely “network trouble.” At the model level it says the implementation refuses to claim a glued global section when it can no longer prove completeness.

That is good protocol design.

9.4 A concrete bad trace

Suppose subscription registration happens before snapshot loading:

```
register subscriber
receive event 42
start snapshot query
receive event 43
snapshot query returns cut 42 including event 42
flush buffer [42,43] without cut filtering
```

The client sees event 42 twice.

The local snapshot is valid. The local buffer is valid. Their naive union is not the intended global history.

Compatibility needs the cut rule to remove the duplicate overlap.

Exercises

1. Write the snapshot-plus-suffix protocol as a cover with two local sections and explicit restriction maps.
2. Construct a missing-event race if the subscriber is registered only *after* snapshot loading.
3. Construct a duplicate-event race if buffered events are not filtered against the snapshot cut.
4. State exactly what “unique gluing” should mean for client-visible UI events when different event histories can project to the same UI state.
5. Modify the model to account for an explicit Overflow outcome. Is the result still best modeled as an ordinary sheaf of sets, or would a richer fiber type be more honest?

10. Atomic projection progress as descent data

The Garden identifies a key open obligation: event append, entity apply, projection cursor advancement, and live fanout are separate boundaries.

Consider three contexts:

E = event-store prefix evidence ,

M = materialized timeline evidence ,

P = projection-checkpoint evidence .

The intended semantic statement is:

A checkpoint at n means that every canonical event through n has been interpreted by that projector and its promised durable materialization is present.

This is not a statement about any one local object. It is a relation among all three.

10.1 Locally valid, globally impossible combinations

Suppose we observe:

EventStore cursor = 51

Timeline materialized = through 51

Projection checkpoint = 50

This can happen if entity apply succeeds and checkpoint advancement fails.

Or:

EventStore cursor = 51

Timeline materialized = through 50

Projection checkpoint = 51

which is more dangerous: the checkpoint claims work not reflected in the durable view.

Each field can contain a syntactically valid monotone ordinal. The global tuple violates the semantic descent condition.

10.2 Transaction boundaries as contexts of joint truth

If apply and cursor advance run in one transaction, there is a larger context U_{MP} in which their joint state is observed atomically.

The projections

$$F(U_{MP}) \rightarrow F(M), F(U_{MP}) \rightarrow F(P)$$

now originate from a single witness.

The architectural value is not just “ACID is good.” It is that the implementation creates the context required for the semantic gluing statement you want to make.

10.3 Descent mindset

In geometry, descent asks whether objects defined locally, together with compatibility data on overlaps, arise from a global object. In software, the analogous question is:

Given durable facts in several subsystems and proofs that they agree where they overlap, do they descend from one valid transaction/execution?

This perspective helps separate:

- data values;
- pairwise compatibility evidence;
- existence of a common global witness.

Exercises

1. Give four possible tuples (eventCursor, materializedThrough, projectionCursor). Classify each as coherent or incoherent under a contract you state precisely.
2. Design restriction maps from a hypothetical ProjectionCommit record to each local context.
3. Compare two repairs: one SQL transaction versus an explicit recovery state machine. How does each provide global-witness information?
4. Explain why UI fanout should probably not be inside the same durable gluing requirement.

11. Sheafification as repair: add the missing local-to-global information

Given a presheaf that fails the sheaf condition, mathematics has a construction called **sheafification** that maps it to an associated sheaf while preserving its local behavior in a precise sense.

You do not need the full construction yet. The software-engineering analogy is already valuable:

Sometimes a raw interface exposes local observations that do not glue. A repair layer can enrich, quotient, or normalize those observations until compatible local data has a canonical global meaning.

Examples of software “sheafification-like” moves include:

- adding stable event identity;
- attaching schema/version coordinates;
- normalizing cursor semantics;
- turning implicit failure into an explicit sum type;
- adding a transaction fence;
- recording a common commit token;
- deduplicating redelivery by semantic identity;
- replacing mutable ambient metadata with versioned metadata carried in the event context.

These are analogies, not claims that every such layer is literally the mathematical sheafification functor.

11.1 A useful design sequence

When gluing fails, diagnose in this order:

1. **Wrong context?** Hidden variables are missing.
2. **Wrong restriction?** Different forgetting paths disagree.
3. **Wrong cover?** Your components do not jointly observe enough.
4. **Insufficient overlap data?** Pairwise interfaces do not carry the shared coordinate needed for compatibility.
5. **No common witness?** You need an atomic record, transaction, log entry, or proof object.
6. **True global obstruction?** Only after the above should you reach for cohomological language.

This sequence prevents category theory from becoming an all-purpose metaphor.

Exercises

1. Classify each open SessionStream obligation under the six diagnostic questions above.
2. Propose a sheafification-like repair for stable redelivery identity.
3. Propose one for deterministic replay.
4. Give a case where quotienting information, rather than adding information, could make gluing unique.

Part IV - Presheaf topoi: a universe of context-dependent software objects

12. The category of presheaves is itself a rich world

Once you fix a context category C , all presheaves

$$C^{op} \rightarrow Set$$

and natural transformations between them form a category

$$\hat{C} = Set^{C^{op}}.$$

This category is a **presheaf topos**.

The word *topos* matters because $\hat{C}$ behaves in many respects like a generalized universe of sets, except its “sets” vary with context.

For software, this is a profound shift:

Instead of one global type T , consider a context-indexed type whose available values and truths vary with what is known.

12.1 Session-dependent types

A presheaf `VisibleEntity` might assign to every context U the entities visible there.

A presheaf `CanReplay` might assign evidence that replay is valid in a given context.

A presheaf `AuthorizedCommand` could vary with tenant/session/principal context.

A natural transformation between such presheaves is a context-respecting program.

This resembles dependency injection and capability systems, but the mathematical discipline is stronger: every context restriction must commute.

12.2 Representables and the Yoneda viewpoint

For an object $U \in C$, the representable presheaf

$$yU = Hom_C(-, U)$$

assigns to context V the set of ways V maps into U .

Yoneda says that maps

$$yU \Rightarrow F$$

correspond naturally to elements of $F(U)$.

Software intuition:

To know a context/object is to know how every test context can map into it; and supplying a context-respecting interpretation out of all probes into U is equivalent to supplying one value over U .

This is close to interface-based reasoning: an object's meaning is revealed by its interactions with all admissible probes. Yoneda is much more exact than that slogan, but the slogan is a good entry point.

12.3 Why this matters for architecture tests

Imagine a “probe category” of test contexts:

- replay prefix;
- snapshot load;
- concurrent live event arrival;
- retry;
- schema-version transition.

A semantic object is trustworthy when all probes and their compositions behave coherently. Yoneda encourages a testing philosophy:

Specify by observable morphisms rather than private representation.

Exercises

1. For the prefix poset, explicitly compute the representable presheaf $y(n)$ at cuts $0, 1, \dots, n+2$.
2. Interpret a natural transformation $y(n) \Rightarrow F$ for a snapshot presheaf.
3. How is property-based testing “probe-like” but not automatically Yoneda reasoning?
4. Define a presheaf of valid commands over authorization contexts. What should restriction mean?

13. Subobjects and local truth

Goldblatt introduces topoi through subobjects and a subobject classifier Ω . We only need the part that changes how you think about software predicates.

In Set, a subset $A \subseteq X$ corresponds to a characteristic function

$$\chi_A: X \rightarrow \{0, 1\}.$$

In a general topos, a subobject classifier

$$\Omega$$

plays the role of an object of truth values.

In a presheaf/sheaf topos, truth can depend on context. It need not be a single global Boolean known everywhere.

13.1 “True at this cut” versus “globally true”

Consider the proposition:

projection timeline is caught up.

At event prefix n , the proposition may be supported. At a later prefix $n+10$, it may no longer be supported until the projector catches up again.

Or consider:

this event has been durably materialized.

A UI fanout observer may not have enough evidence to assert it. A database transaction context might.

Topos logic turns this dependency on context into mathematics rather than treating it as epistemic hand-waving.

13.2 Intuitionistic flavor

In many sheaf/presheaf settings, logic is intuitionistic. Failure to establish P does not automatically establish $\neg P$.

Software engineers already live with this:

- unknown is not false;
- absence from a cache is not proof of nonexistence;
- failure to observe a commit is not proof it did not commit;
- lack of authorization evidence is not the same proposition as explicit denial, though secure systems may deliberately map unknown to deny at a policy boundary.

The topos perspective says some of this three-valued-looking behavior is better understood as **contextual truth** rather than merely adding a third Boolean.

13.3 Kripke-Joyal intuition without full semantics

Forcing semantics in a sheaf topos interprets statements relative to a stage/context U . Roughly:

$$U \Vdash \varphi$$

means φ is valid when reasoning from context U , with stability under restriction to smaller contexts.

For SessionStream, you might read

$$U_n \Vdash \text{“snapshot is coherent through } n\text{”}$$

as a statement whose evidence belongs to a particular cut/context, not an eternal global Boolean.

This is enough motivation for now. A full internal-language treatment can wait until you want to formalize policies or specifications inside a topos.

Exercises

1. List five SessionStream predicates that should be indexed by context rather than treated as timeless Booleans.
2. Give a predicate whose truth is monotone under restriction to earlier prefixes.
3. Give one that is not monotone under *extension* to later prefixes, explaining why this does not violate presheaf restriction stability.
4. Compare unknown, false, and “not forced at this context.”

Part V - From gluing to cohomology

14. Why cohomology enters only after the sheaf story

Cohomology is not a magic inconsistency detector. It is a systematic way of extracting invariants from chains of local compatibility data, especially when values carry algebraic structure.

The path is:

1. choose a cover/context complex;
2. assign algebraic data to regions and overlaps;
3. form cochains by dimension;
4. define coboundary maps that measure compatibility defects;
5. quotient closed defects by those that are merely changes of local coordinates.

For Čech-style cohomology of a cover $\{U_i\}$, one forms

$$\begin{aligned} C^0 &= \prod_i F(U_i), \\ C^1 &= \prod_{i < j} F(U_i \cap U_j), \\ C^2 &= \prod_{i < j < k} F(U_i \cap U_j \cap U_k), \end{aligned}$$

and so on, when F is valued in abelian groups/modules so subtraction makes sense.

A coboundary

$$\delta^0 : C^0 \rightarrow C^1$$

records disagreements of local values on pairwise overlaps.

Then

$$H^0 = \ker \delta^0$$

captures compatible global-like 0-data, while

$$H^1 = \ker \delta^1 / \Im \delta^0$$

captures closed 1-dimensional discrepancy patterns that cannot be removed by changing local representatives.

14.1 Software translation

- 0-cochain: a choice of local coordinate/value for each component;
- 1-cochain: differences/transition values on pairwise interfaces;
- 2-cochain: compatibility defect on triples;
- coboundary: “take the alternating sum around the boundary”;
- cocycle: locally self-consistent discrepancy data;
- coboundary: discrepancy caused merely by choosing different local origins;
- cohomology class: residual discrepancy not removable by local reparameterization.

The phrase “hole” is shorthand for why such residual circulation can exist.

14.2 Why ordinary application state is not automatically suitable

A TimelineEntity is not an abelian group. A protobuf oneof does not support subtraction. An Overflow error is not a vector.

So begin with a set-valued sheaf for semantic correctness. Introduce cohomology only for selected *linearizable diagnostics* such as:

- ordinal offsets;
- version deltas;
- parity bits;
- counts;
- conservation quantities;
- checksums in an abelian group;
- additive clock skew estimates;
- permission-difference lattices after suitable algebraization.

Exercises

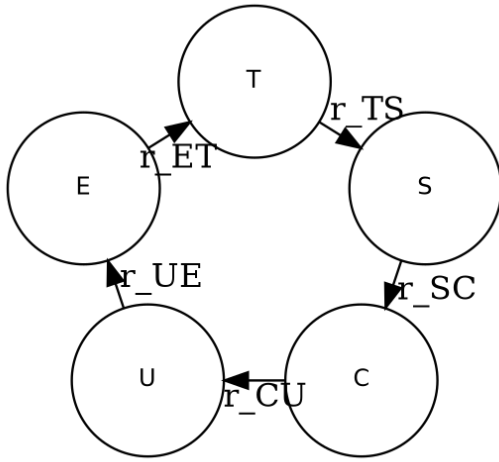
1. Explain why “event payloads form a group” is usually a bad modeling assumption.
2. Identify three numeric/additive quantities in SessionStream that could support a cochain model.
3. What information is lost when you replace rich semantic state by an ordinal-offset diagnostic?
4. Why can a nonzero obstruction be useful even if vanishing cohomology does not prove the whole software system correct?

15. A complete H1 example: ordinal semantics around a cycle

Consider five observation contexts:

- E : EventStore;
- T : timeline projection;
- S : snapshot;
- C : client;
- U : live UI stream.

Suppose each uses an integer coordinate $x_i \in \mathbb{Z}$ intended to represent “where we are” in the session history.



A cycle of pairwise cursor translations

On every edge define a transition offset

$$r_{ij} = x_j - x_i.$$

A choice of local coordinates $(x_E, x_T, x_S, x_C, x_U)$ is a 0-cochain.

The edge offsets r form a 1-cochain.

If the offsets arise from real local coordinates, their sum around the cycle telescopes:

$$r_{ET} + r_{TS} + r_{SC} + r_{CU} + r_{UE} = 0.$$

15.1 The off-by-one obstruction

Suppose four adapters agree on “last included ordinal” but one adapter interprets its integer as “next ordinal to consume.”

You might obtain:

$$r_{ET} = 0, r_{TS} = 0, r_{SC} = 0, r_{CU} = 1, r_{UE} = 0.$$

Then

$$\sum r = 1.$$

No assignment of local origins x_i can generate these transition equations simultaneously.

The error is global in the sense that every individual interface can appear plausible while the composition around the loop returns shifted.

This is close to a nontrivial 1-dimensional circulation.

15.2 Gauge change / local reparameterization

Suppose component i changes its local origin by a_i :

$$x_i' = x_i + a_i.$$

Then edge offsets change by

$$r_{ij}' = r_{ij} + a_j - a_i.$$

This is exactly adding a coboundary δa .

Such a change may alter individual edge numbers but cannot change the total circulation around a closed loop.

That quotient idea is the heart of cohomology:

ignore discrepancies that are artifacts of local coordinate choices; retain discrepancies invariant under all local rebasings.

15.3 Tiny matrix calculation

Orient the five cycle edges. The incidence/coboundary matrix D taking vertex values to edge differences is

$$D = \begin{pmatrix} -1 & 1 & 0 & 0 & 0 \\ 0 & -1 & 1 & 0 & 0 \\ 0 & 0 & -1 & 1 & 0 \\ 0 & 0 & 0 & -1 & 1 \\ 1 & 0 & 0 & 0 & -1 \end{pmatrix}.$$

For a vertex vector x , the edge differences are

$$r = D x.$$

Every vector in $\Im D$ has edge sum zero. Therefore an edge vector with total sum 1 cannot be a coboundary.

For a simple cycle with no filled 2-cell, there is one independent circulation direction, so H^1 has rank one over a field (or behaves like Z with integer coefficients).

If you fill the cycle with enough 2-dimensional compatibility data, the first cohomology can vanish. Geometrically: the hole is gone. Architecturally: the higher-order witness constrains circulation that pairwise interfaces alone allowed.

Exercises

1. Verify directly that every Dx has edge sum zero.
2. Solve $Dx=r$ for $r=(0,0,0,1,0)^T$. Explain the inconsistency.
3. Change two local origins and compute the resulting edge vector. Verify that total circulation is unchanged.
4. Interpret “filling the cycle” as an added multi-component invariant check. What concrete witness could SessionStream add?
5. Replace Z with $Z/2Z$. What kind of software diagnostic might parity detect?

16. Higher-dimensional consistency: triangles are not decoration

Pairwise compatibility is only the beginning.

Suppose three components A, B, C all overlap. Pairwise translators

$$t_{AB}, t_{BC}, t_{AC}$$

should satisfy a triple compatibility condition:

$$t_{AC} = t_{BC} \circ t_{AB}$$

on the common region.

If this law holds, the triangle can be regarded as coherently filled. If not, walking $A \rightarrow B \rightarrow C$ disagrees with $A \rightarrow C$.

Software examples include:

- schema migrations across three versions;
- coordinate transforms across three services;
- authorization scopes translated through two gateways versus directly;
- event-to-timeline-to-client interpretation versus event-to-UI-to-client interpretation;
- snapshot version, event schema version, and browser decoder version.

A 2-cocycle can encode triple-overlap defects in suitable algebraic settings. Higher cohomology continues the same pattern: consistency conditions among consistency conditions.

16.1 Why dimensions appear naturally in architecture

Whenever you say:

- “these two systems agree” - edge;
- “these three translations compose coherently” - face;
- “these four triple agreements themselves agree” - 3-simplex;

there is a simplicial hierarchy.

This does *not* mean every microservice graph should be turned into a simplicial complex. It means that when overlapping contexts and higher-order coherence are central, simplicial language is a natural compression.

Exercises

1. Find a SessionStream triangle where two paths from canonical event to client-visible observation should commute.
2. State a triple-overlap law involving event schema, persisted payload, and browser decoder.
3. Give a case where all three pairwise relations hold but a chosen family of translations is not coherently associative.
4. Research project: encode a small architecture as a simplicial complex and compute Betti numbers. Explain what the numbers do *not* prove about software correctness.

Part VI - A formal SessionStream study

17. Build the SessionStream context category

We now assemble a concrete mathematical laboratory.

Define a context as a tuple

$$U = (s, n, k, v, w)$$

where:

- s : session identity;
- n : event-prefix cut;
- k : observer kind (events, timeline, snapshot, ui, client, or selected products);
- v : schema/version environment;
- w : witness strength, such as local-read, transactional-read, committed, or client-observed.

This is intentionally richer than the production API. The mathematical base is allowed to expose coordinates that production code hides.

17.1 Order/restriction relation

Say $V \leq U$ when V can be obtained by lawful information loss:

- same session;
- earlier or equal cut;
- no stronger observer set;
- compatible schema restriction;

- no stronger witness claim.

Then C is a poset category if there is at most one restriction arrow between contexts.

You might later need a non-posetal category when there are multiple meaningful translations between the same contexts. Starting with a poset keeps the first model tractable.

17.2 Define the semantic presheaf

Let $F(U)$ be the set of assignments that satisfy all invariants visible inside U .

A section may contain fields such as:

```
sessionId
cut
eventDigest
eventStoreCursor
projectionCursor[timeline]
snapshotOrdinal
maxEntityLastEventOrdinal
materializationDigest
uiBatchOrdinals
clientStateDigest
schemaVersion
witnessToken
```

Not every context contains every field.

Restriction drops fields and possibly truncates data to an earlier cut.

17.3 Local validity predicates

Examples:

Prefix validity

$$1 \leq \text{Ordinal}(e_1) < \dots < \text{Ordinal}(e_n).$$

Snapshot cut

$$\text{LastEventOrdinal}(x) \leq \text{SnapshotOrdinal}$$

for each entity x .

Projection checkpoint

$$\text{ProjectionCursor} = n \Rightarrow \text{MaterializedThrough} \geq n.$$

Live suffix

$$\forall b \in \text{LiveBatches}, \text{Ordinal}(b) > \text{SnapshotOrdinal}.$$

Deterministic replay

$$\mathit{rebuild}(S_0, H) = \mathit{liveFold}(S_0, H)$$

under equal explicit coordinates.

17.4 Covers to study

Use several different covers rather than forcing one giant topology.

Reconnect cover

$$\{\mathit{SnapshotView}, \mathit{LiveSuffixView}\}$$

covering `ClientHistory`.

Projection commit cover

$$\{\mathit{EventStoreView}, \mathit{MaterializationView}, \mathit{ProjectionCursorView}\}$$

covering `DurableProjectionState`.

Replay cover

$$\{\mathit{InitialView}, \mathit{EventPrefix}, \mathit{ProjectorVersion}, \mathit{MetadataVersion}\}$$

covering `RebuildResult`.

Cross-runtime schema cover

$$\{\mathit{GoType}, \mathit{ProtoDescriptor}, \mathit{PersistedBytes}, \mathit{JSDecoder}\}$$

covering `SemanticMessage`.

These covers ask different local-to-global questions. A topos/sheaf model is useful precisely because “context” can be adapted to the semantic problem.

Exercises

1. Formalize the partial order on (s, n, k, v, w) . Check reflexivity, antisymmetry, and transitivity.
2. Define five explicit restrictions and prove two nontrivial composition equations.
3. For each cover above, list its pairwise overlap data.
4. Which cover most directly exposes the current SQLite consistent-cut obligation?
5. Which cover exposes hidden projector inputs?

18. Turn traces into sections

The Architecture Garden verification work already treats runtime traces as evidence. That makes traces a natural bridge into presheaf tooling.

Suppose one test execution emits records:

```
EventAppended(session=s, ordinal=40)
EntityApplied(session=s, entity=x, last=40)
ProjectionCursorAdvanced(session=s, projector=timeline, ordinal=40)
SubscriberRegistered(session=s)
SnapshotLoaded(session=s, cut=40)
UIBatchBuffered(session=s, ordinal=41)
SnapshotSent(session=s, cut=40)
UIBatchSent(session=s, ordinal=41)
```

A **trace interpreter** maps these raw records to local sections:

$$I_U: \text{Trace} \rightarrow F(U).$$

For every context U , the interpreter extracts only the facts observable there.

18.1 Naturality as an instrumentation requirement

Ideally trace interpretation commutes with restriction:

$$I_V(\text{trace}) = \rho_V^U(I_U(\text{trace})).$$

If your low-level and high-level observers disagree after projection, instrumentation itself is not a faithful witness.

This gives a principled specification for telemetry adapters.

18.2 Runtime gluing checker

A research tool could:

1. ingest a recorded trace;
2. construct sections for configured contexts;
3. check local predicates;
4. restrict sections to overlaps;
5. report incompatibilities;
6. attempt a global-section reconstruction;
7. compute optional cohomological diagnostics for linearized quantities.

Pseudo-API:

```
type ContextID string
```

```
type Section map[string]Value
```

```
type Presheaf interface {
    Restrict(from, to ContextID, s Section) (Section, error)
```

```

    Validate(ctx ContextID, s Section) error
}

```

```

type Cover struct {
    Global ContextID
    Local []ContextID
}

```

```

type GlueReport struct {
    LocalErrors []Violation
    OverlapErrors []Violation
    Global *Section
    Ambiguous bool
}

```

The point is not that this Go interface is a sheaf. It is a harness for experimenting with one finite model.

Exercises

1. Extend GlueReport to distinguish no-extension from multiple-extension failures.
2. Design an overlap error for a duplicate snapshot/live event.
3. How would you represent an explicit overflow outcome without making it look like ordinary data?
4. Define a property test: generated globally valid traces should restrict to pairwise compatible local sections.
5. Define the reverse property. Is pairwise compatibility sufficient for global validity in your model? Find a counterexample if not.

19. An API-design method based on local-to-global sufficiency

The same framework applies outside SessionStream.

Given a transactional invariant I :

1. list every semantic variable on which I depends;
2. define contexts corresponding to request parameters, server lookups, credentials, current version, and transaction witness;
3. define restriction maps between richer and poorer contexts;
4. calculate the fiber of possible global completions compatible with a request;
5. test whether I is constant on that fiber;
6. if not, add a parameter, authoritative lookup, version coordinate, or stronger witness;
7. when several services supply pieces, define a cover and gluing conditions;
8. only introduce cohomology if overlap discrepancies have useful algebraic structure.

19.1 Example: optimistic concurrency

API:

PATCH /orders/{id}

If-Match: version=17

Invariant:

update applies to exactly the order state the caller reviewed.

Contexts:

- caller state $(id, version, patch)$;
- authoritative state $(id, currentVersion, currentData)$;
- committed transaction $(id, oldVersion, newVersion, patchDigest)$.

Without version, the request fiber contains completions corresponding to many intervening states. The invariant is not constant over the fiber.

Adding the version and performing compare-and-swap is not merely “adding a field.” It changes the local-to-global problem so that request evidence can glue to exactly one committed predecessor state.

19.2 Example: idempotency key

A payment API with retries needs a stable logical-operation coordinate.

Without an idempotency key, two identical HTTP bodies at different times may legitimately mean two payments. Content equality cannot define semantic identity.

Adding

IdempotencyKey

creates an overlap coordinate across retry contexts. The server can now ask whether multiple deliveries are local views of one global operation.

This is directly analogous to the open stable-redelivery-identity question in SessionStream.

Exercises

1. Apply the eight-step method to a bank transfer endpoint.
2. Apply it to a deployment API that must update code and schema atomically.
3. Apply it to a cache invalidation API where stale values are permitted for 30 seconds.
4. Identify which problems are solved by adding coordinates and which require stronger atomic witnesses.

20. A research program for SessionStream

The mathematical machinery becomes valuable when it suggests experiments or architectural hardening. Here is a staged program.

Stage 1 - Presheaf specification

Write a machine-readable context graph for:

- canonical events;
- timeline state;
- projection cursors;
- snapshots;
- live UI batches;
- client-visible reconstruction.

For every edge, implement a restriction function and property-test composition.

Deliverable: a finite presheaf model over selected trace summaries.

Stage 2 - Cover/gluing tests

Encode covers for:

- snapshot + suffix;
- event + materialization + projector checkpoint;
- replay inputs;
- cross-runtime schema identity.

Generate compatible and incompatible traces.

Deliverable: counterexample-producing global-section checker.

Stage 3 - Transactional witnesses

Compare architecture variants:

- current separate operations;
- transactional projection apply + cursor advance;
- read-transaction snapshots;
- stable event IDs.

Measure which previously non-gluable traces become impossible.

Deliverable: an invariant matrix tied to implementation variants.

Stage 4 - Linear diagnostic sheaf

Define integer-valued quantities:

event cursor
projection cursor
snapshot cut
max entity last ordinal
client acknowledged cut

Define transition offsets and compute cycle circulation.

Deliverable: coboundary matrix and H^0/H^1 for the finite architecture complex.

Stage 5 - Fault injection

Mutate:

- last included versus next to consume semantics;
- dropped buffered event;
- duplicated buffered event;
- cursor advance before apply;
- nondeterministic projector input;
- reused versus regenerated retry ID.

Ask which layer catches each mutation:

- local validator;
- overlap check;
- global-section search;
- cohomological diagnostic;
- none.

Deliverable: a map from bug class to mathematical witness.

Stage 6 - Internal logic experiments

Only after the above, model context-dependent propositions such as:

- “timeline is caught up”;
- “snapshot is safe to expose”;
- “client has complete history through n”;
- “this command is admissible under the current schema and authority context.”

Explore what these predicates look like as subobjects in the presheaf/sheaf topos.

Deliverable: a small executable logic of evidence, not a rewrite of business logic in categorical terms.

Part VII - Exercises, projects, and selected solutions

21. Cumulative problem set A: categorical foundations

A1. Universal property of a consistent view

Let E be event histories, T timeline materializations, and C cursor values. Define maps

$$f:T \rightarrow C, g:E \rightarrow C.$$

Interpret the pullback $T \times_C E$ as consistent event/materialization pairs.

1. Choose concrete definitions for f and g .
2. State what information is lost if C contains only an ordinal.
3. Add a digest coordinate and compare the new pullback.

A2. Equalizer-based replay check

Define

$$replay, live: E^i \rightarrow T.$$

Construct their equalizer. Explain what it proves and what it does not prove about hidden external effects.

A3. Naturality of encoding

Let $F(n)$ be protobuf snapshot state and $G(n)$ browser snapshot state. Define restriction by truncation to $m \leq n$. A decoder family $d_n: F(n) \rightarrow G(n)$ is natural if

$$d_m \circ restrict_{n,m}^F = restrict_{n,m}^G \circ d_n.$$

Construct a bug that violates this square.

22. Cumulative problem set B: presheaves and sheaves

B1. Build a finite presheaf by hand

Use four contexts:

$U_0 = \{\text{sessionId}\}$

$U_1 = \{\text{sessionId}, \text{snapshotOrdinal}\}$

$U_2 = \{\text{sessionId}, \text{liveOrdinal}\}$

$U_3 = \{\text{sessionId}, \text{snapshotOrdinal}, \text{liveOrdinal}\}$

Let $F(U)$ be assignments satisfying local type/range checks.

1. List all restriction maps.
2. Verify composition.
3. Add the law $\text{liveOrdinal} > \text{snapshotOrdinal}$ only at U_3 .
4. Explain why sections over U_1 and U_2 may each be locally valid but fail to glue over U_3 .

B2. Under-determined gluing

Modify U_3 to contain an extra `schemaVersion` invisible to U_1 and U_2 . Show that compatible local data can have multiple global extensions.

What extra overlap/context would restore uniqueness?

B3. Snapshot race analysis

Create four traces:

1. correct hydration;
2. missing event;
3. duplicate event;
4. overflow.

For each, provide local sections on `SnapshotView` and `LiveView`, their overlap restrictions, and whether a global client history exists.

23. Cumulative problem set C: cohomology by calculation

C1. Three-node cycle

Take vertices A, B, C and oriented edges $A \rightarrow B, B \rightarrow C, C \rightarrow A$. Construct

$$D = \begin{pmatrix} -1 & 1 & 0 \\ 0 & -1 & 1 \\ 1 & 0 & -1 \end{pmatrix}.$$

1. Find the rank and nullspace of D over R .
2. Describe H^0 .
3. Show that edge vector $(1, 1, 1)$ is not in $\Im D$.
4. Interpret it as a software circulation.

C2. Fill the triangle

Add one 2-simplex whose boundary is the cycle.

The next coboundary δ^1 measures total oriented circulation around the triangle.

1. What 1-cochains are cocycles now?

2. Explain intuitively why the previous H^1 disappears over a field.
3. Translate this into adding a three-way compatibility witness.

C3. Real SessionStream offsets

Instrument a test run with:

EventStoreCursor
TimelineCursor
SnapshotOrdinal
ClientAppliedOrdinal

Choose an architecture graph and define edge differences. Inject one off-by-one semantic mismatch. Determine whether it produces nonzero circulation.

24. Capstone projects

Project 1 - The SessionStream Sheaf Lab

Build a Go package or notebook containing:

- finite context category;
- sections represented by typed records;
- restriction maps;
- cover definitions;
- compatibility checker;
- brute-force global extension search for small fibers;
- counterexample printer.

Success criterion: reproduce one known good hydration trace and at least three intentionally broken traces.

Project 2 - Transaction topology comparison

Model two architectures:

- A. entity apply and projection cursor are separate operations;
- B. they commit atomically.

Generate all small transition traces up to some bound. Compare the set of local section families each architecture can produce.

Question: which incompatible families disappear in B?

Project 3 - Event identity refinement

Add stable EventId to the mathematical model before touching production code.

Model retries as multiple delivery contexts restricting to one logical-event context.

Ask whether duplicate deliveries now glue uniquely to one accepted-event global section.

Then determine what store constraint or idempotency table would make the model executable.

Project 4 - Cohomological cursor linter

Represent cursor semantics between components as affine integer translations.

Compute cycle sums automatically. Report a minimal cycle whose sum is nonzero.

This is a realistic small tool: unlike full semantic cohomology, it can detect incompatible last seen, next, inclusive, and exclusive coordinate conventions.

Project 5 - Contextual truth dashboard

Define evidence-valued predicates:

SnapshotCoherent(session,n)

ProjectionCaughtUp(session,projector,n)

ClientCompleteThrough(session,n)

ReplayDeterministic(session,n,version)

Record the contexts that can force each predicate. Render an evidence graph showing where a proposition is known, unknown, or refuted by explicit counterevidence.

Do not call this a topos implementation unless you actually implement the categorical structure. Treat it as an experiment motivated by internal logic.

25. Selected solutions and solution sketches

Solution sketch: Exercise 2.3

A possible race:

t0: read projection cursor -> 42

t1: concurrent apply event 43, entity x now has LastEventOrdinal=43

t2: read entity rows -> includes x@43

return SnapshotOrdinal=42 with x@43

Each read can be a valid SQLite query result at its own instant. The pair fails the intended snapshot predicate

$$LastEventOrdinal(x) \leq SnapshotOrdinal.$$

A read transaction makes both reads projections of one database snapshot, giving the common witness needed by the semantic model.

Solution sketch: Exercise 6.3

An API might provide (orderId, requestedRefundAmount). Many global states can differ in shipping address, display name, or unrelated item metadata. If the refund invariant depends only on captured amount and already-refunded amount, then the parameters plus authoritative lookup may determine the invariant without determining the full order state.

Solution sketch: Exercise 9.2

Bad ordering:

start loading snapshot
(event 50 occurs and is fanned out)
finish loading snapshot at cut 49
register subscriber

Event 50 is neither represented in the snapshot nor observed live. The cover has a gap. Register-before-load plus buffering closes that gap.

Solution sketch: Exercise 15.1

Each row of D contributes one $-x_i$ and one $+x_j$. Summing all rows around the closed cycle cancels every vertex value once positively and once negatively. Therefore

$$1^T D x = 0.$$

Any edge vector whose total is nonzero is outside $\Im D$.

Solution sketch: B2

If schemaVersion lives only in the global context, then two globals differing only by schema version restrict to the same two locals. Gluing is not unique. You can restore uniqueness by adding schema version to at least enough local/overlap contexts that compatible locals determine it, or by quotienting the global semantics so schema versions that decode identically are intentionally considered equivalent.

26. Glossary for software engineers

Arrow / morphism. A structure-preserving relationship chosen by your model. Do not assume it means a function call.

Category. Objects and composable arrows with identity and associativity laws.

Cone. One object mapping compatibly to every object of a diagram.

Limit. Universal compatible cone. Think “best joint witness satisfying an entire diagram.”

Product. Universal way to retain two independent views.

Equalizer. Universal subobject on which two arrows agree.

Pullback. Universal compatible pair over shared information; a typed join with a witness.

Functor. Structure-preserving translation between categories.

Natural transformation. Context-independent translation between functors; all structure-respecting squares commute.

Opposite category. Reverse every arrow. Presheaves use this because restrictions move opposite to inclusions.

Presheaf. Context-indexed data with coherent restriction/forgetting maps.

Section. One value/assignment over a context.

Restriction. Forget or localize a section to a smaller context.

Cover. Local contexts intended jointly to account for a larger one.

Nerve. Simplicial shape recording which cover pieces overlap jointly.

Sheaf. Presheaf in which compatible local sections glue uniquely.

Global section. A coherent value over the whole context.

Stalk / germ. Ways of focusing sections to infinitesimal/local behavior around a point in topological sheaf theory. For software, useful later when studying “what is known at one context point,” but not required for the first SessionStream model.

Topos. A category with set-like structural features; presheaf categories and suitable sheaf categories are central examples.

Subobject classifier. Generalized object of truth values classifying subobjects.

Internal logic. Logic interpreted using the structural operations of a category/topos rather than an external set universe.

Cochain. Assignment of algebraic data to cells/overlaps of a given dimension.

Coboundary. Operator measuring the boundary-wise change of a cochain.

Cocycle. Cochain whose next coboundary vanishes.

Coboundary class. A discrepancy explainable by changing lower-dimensional local choices.

Cohomology. Cocycles modulo coboundaries; residual global discrepancy/structure not removable by local reparameterization.

27. Reading map back into Goldblatt

This custom text intentionally reorders Goldblatt. Use the following map when returning to the original book.

This text	Goldblatt sections to consult	Why
Chapters 1-3	§§3.3-3.16	Isomorphisms, initial/terminal objects, duality, products, equalizers, limits, pullbacks, exponentials
Chapter 4	§§9.1-9.3	Functors, natural transformations, functor categories
Chapters 5-11	§14.1 especially; also §4.5	Stacks/presheaves, restriction, compatibility/gluing, sheaves
Chapter 12	§9.3 and general topos chapters	Functor categories and presheaf categories
Chapter 13	§§4.1-4.3, 7, 8, 14.5-14.6	Subobjects, classifiers, intuitionistic/local truth, Kripke-Joyal semantics
Universal-construction revisits	§§15.1-15.2	Adjunctions and preservation of limits/colimits

A notable difference is pedagogical order. Goldblatt can postpone functors because he develops much of elementary topos theory first and introduces functor categories later. For a sheaf-first software route, functors must arrive earlier because a presheaf is literally a contravariant functor.

When you reach Goldblatt §14.1, pay special attention to three moves:

1. a stack/presheaf assigns sets to open regions and restriction maps to inclusions;
2. compatible local sections are required to agree on overlaps;
3. the sheaf condition states that every compatible local family has exactly one global section, and this can be restated as a limit property.

Those are the exact moves this book has reinterpreted as software architecture.

28. Source and scope notes

Primary pedagogical source

Robert Goldblatt, *Topoi: The Categorical Analysis of Logic*. This custom text uses the uploaded edition as a study guide, especially Chapters 3, 4, 9, 14, and 15. Definitions and exercises here are independently written and adapted to the SessionStream case study; they are not reproductions of Goldblatt's prose or problem sets.

SessionStream source

Architecture Garden - sessionstream, PARC:

<https://parc.yolo.scapegoat.dev/note/research/software-architecture-garden/sessionstream/readme>

The Garden snapshot used here identifies:

- module `github.com/go-go-golems/sessionstream`;
- branch `main`;
- analyzed repository commit `fb6b70d62915874e3d3cb9c0b1557814e638ac68`;
- primary implementation under `pkg/sessionstream`;
- the architecture and hardening laws summarized throughout this text.

The codebase may have evolved after that analyzed commit. Treat the mathematical model as a research model of the documented architecture, not as a proof of the current repository.

Mathematical scope

This book deliberately stops short of several subjects that would become relevant later:

- Grothendieck topologies in full generality;
- sheafification construction and its adjunction;
- derived functors and general sheaf cohomology;
- spectral sequences;
- higher category theory and infinity-topoi;
- homotopy type theory;
- categorical semantics of concurrency in full generality.

The next mathematical step after finishing this text should probably be a rigorous treatment of presheaves/sheaves on posets and small categories, followed by Čech cohomology on finite covers/cellular sheaves. Only then is it worth escalating to derived-functor sheaf cohomology.

29. Final mental model

Keep this chain in your head:

An architecture gives many contexts of observation.



A presheaf says what data can live in each context and how it restricts.



A cover says which local contexts are supposed to account for a whole.



A sheaf says compatible local data determines one global state.



A topos is a universe in which these context-varying objects can be reasoned about.



Cohomology can expose algebraic obstruction patterns that survive all local repairs.

For SessionStream specifically:

SessionId gives scope.

Ordinal gives a temporal coordinate, not semantic identity.

Events, projections, snapshots, cursors, and clients are distinct local views.

Restriction means truncating or forgetting view dimensions.

Snapshot + live suffix is a gluing protocol.

Atomicity creates common witnesses needed by stronger gluing claims.

Stable IDs and versions add missing coordinates.

Cursor cycles provide a first computable cohomological laboratory.

If those statements become intuitive rather than merely verbal, the abstract mathematics is starting to become part of your engineering vocabulary.